

Reinforcement Learning: Model-Free Control (ESE 3990)

Instructor:
Antonio Loquercio

Fall 2026
University of Pennsylvania

What We Will Cover in the Next Classes

- Solve a *known* MDP
 - Planning by Dynamic Programming
- Evaluate the value function of an *unknown* MDP
 - Model-free prediction
- Optimize the value function/policy of an *unknown* MDP
 - Model-free control

Why Model-free Control?

- Model-free control is useful when, either:
 - The MDP model is unknown, but experience can be sampled.
 - The MDP model is known but too large to use except by sampling.
- A few examples:
 - Robosoccer
 - LLM finetuning
 - Stock-Portfolio optimization
 - Robot Walking (more or less)

Dense-Fog Golf



$$s \in S$$

- No prior knowledge about the field.
- You can only see the ball's location.



s_0

$$a \in A$$



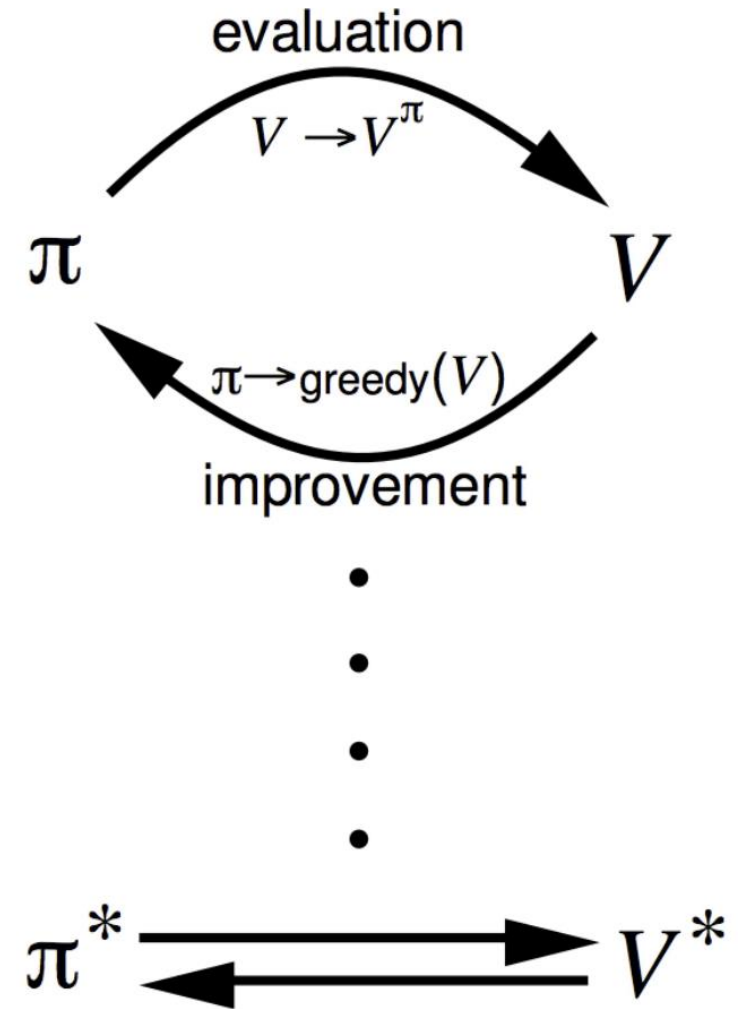
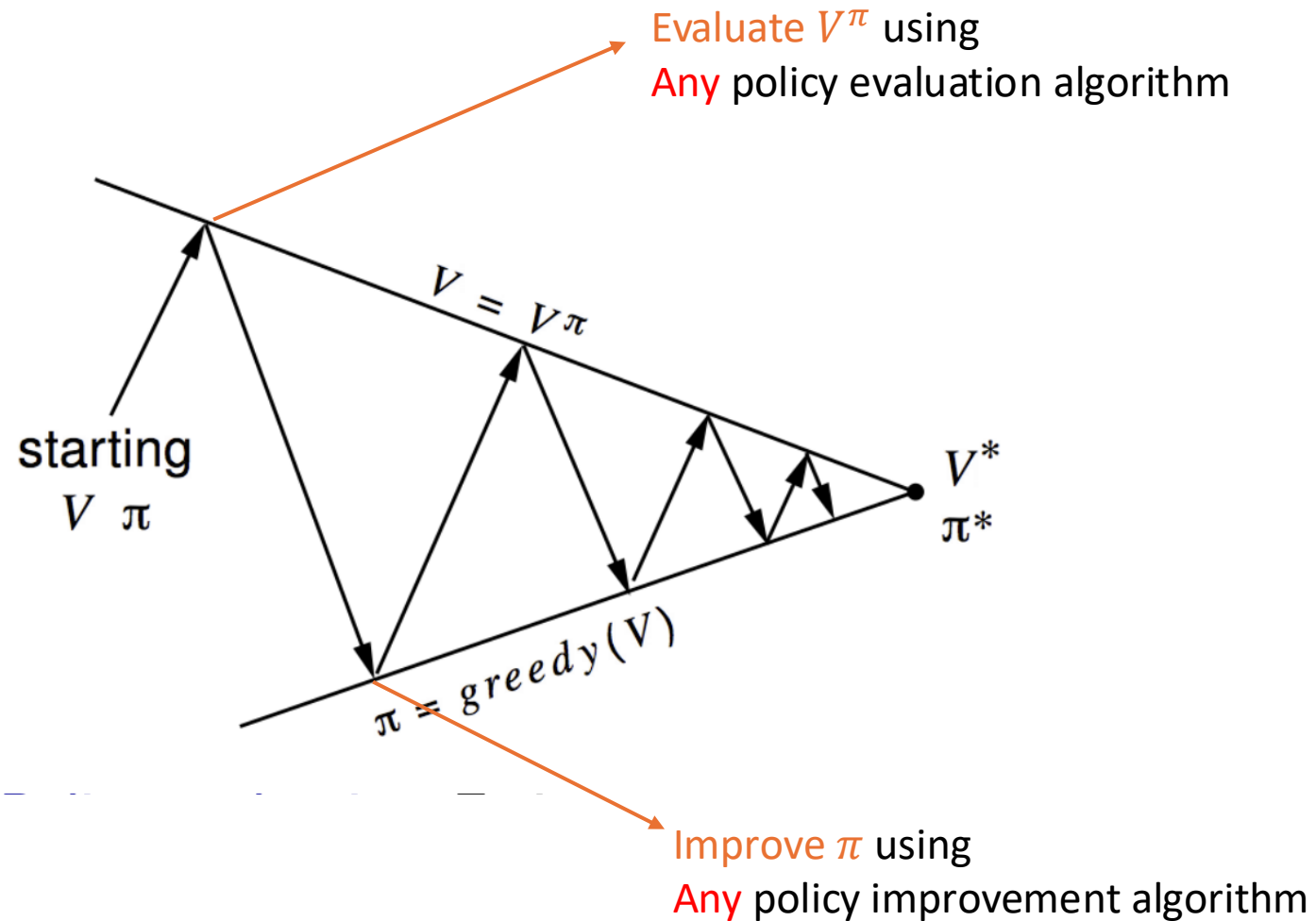
Driver



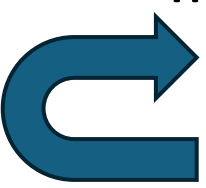
Putt

$$r(s, a) = -1$$

Generalized Policy Iteration (Refresher)



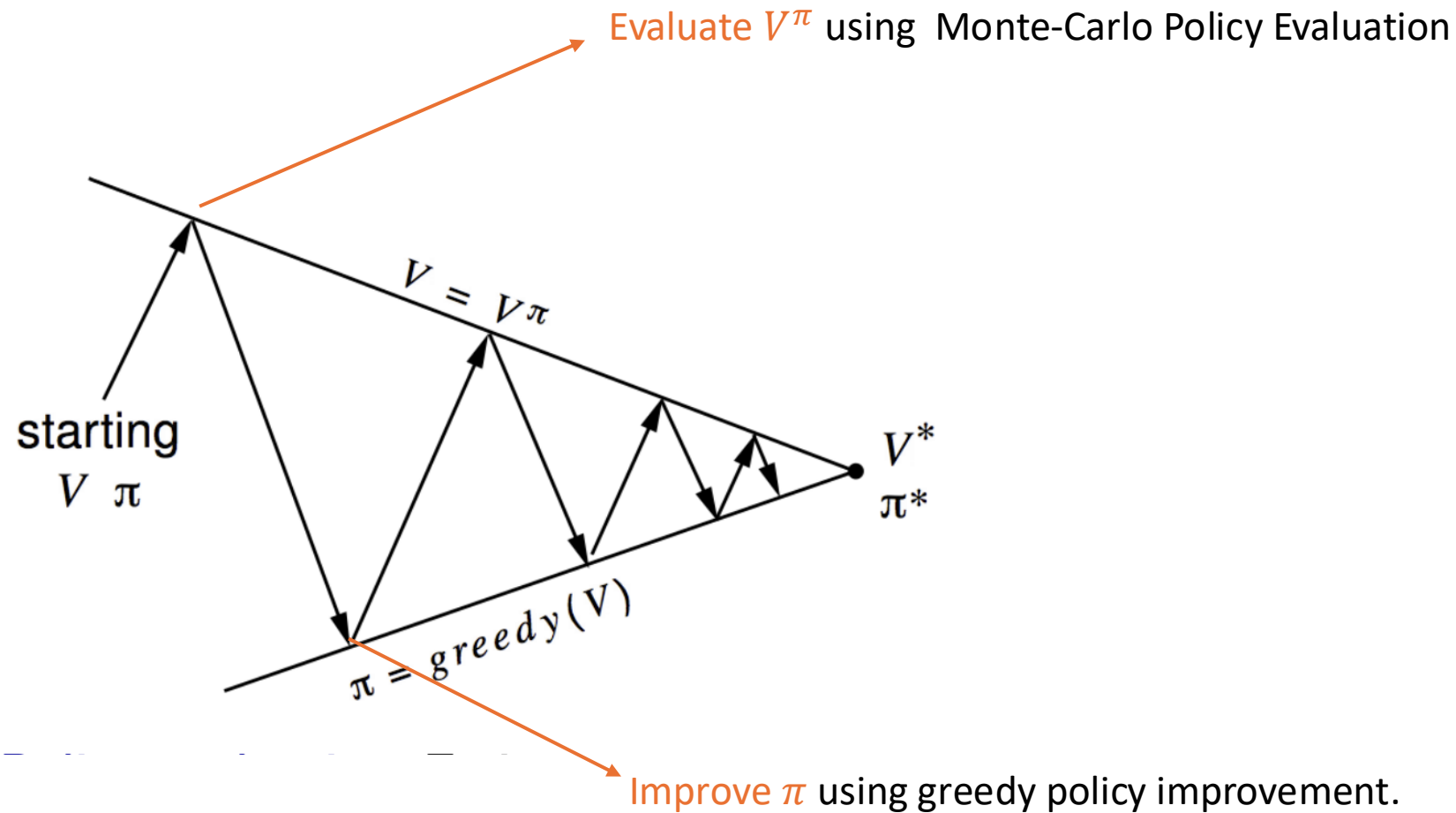
Policy Iteration (Refresher)

- Init with a policy π_0 , then repeat until tired:
- 
- **Evaluate** the policy π (e.g., using the algorithms of the previous lecture)
 - **Improve** the policy by acting *greedily* with respect to V^π

$$\pi'(a|s) = \begin{cases} 1 & \text{if } a = \arg \max_{a'} r(s, a') + \gamma \sum_{s' \in \mathcal{S}} P_{ss'}^{a'} V^\pi(s') \\ 0 & \text{otherwise} \end{cases}$$

- Meaning of *greedy*:
 - Optimize for just the next *step*, not the entire trajectory. After taking the “greedy” action a , we still follow π .

Monte-Carlo Policy Iteration



$$a \in A$$

Does this work in dense-fog golf?

- Let's try to act greedily w.r.t. $V^{putt}(s)$:
- $\pi'(a|s) = \arg \max_{a'} r(s, a') + \gamma \sum_{s' \in S} P_{ss'}^{a'} V^{putt}(s')$
- If $a=\text{driver}$: $-1 + P_{ss'}^{driver} V^{putt}(s')$
- If $a=\text{putt}$: $-1 + P_{ss'}^{putt} V^{putt}(s')$
- Do you see any problem with this?



Driver



Putt

$$r(s, a) = -1$$

$$a \in A$$

Does this work in dense-fog golf?

- Acting greedily w.r.t. a value function requires *predicting* the successor state s' .
- Predicting the next from the current state requires a model (or simulator) of the environment.
- That's why this is called *model-based* control.
- But for today, we assume to have **no** model.
 - This is called model-free control.



Driver



Putt

$$r(s, a) = -1$$

Let's go back to the meaning of *Greedy Action*

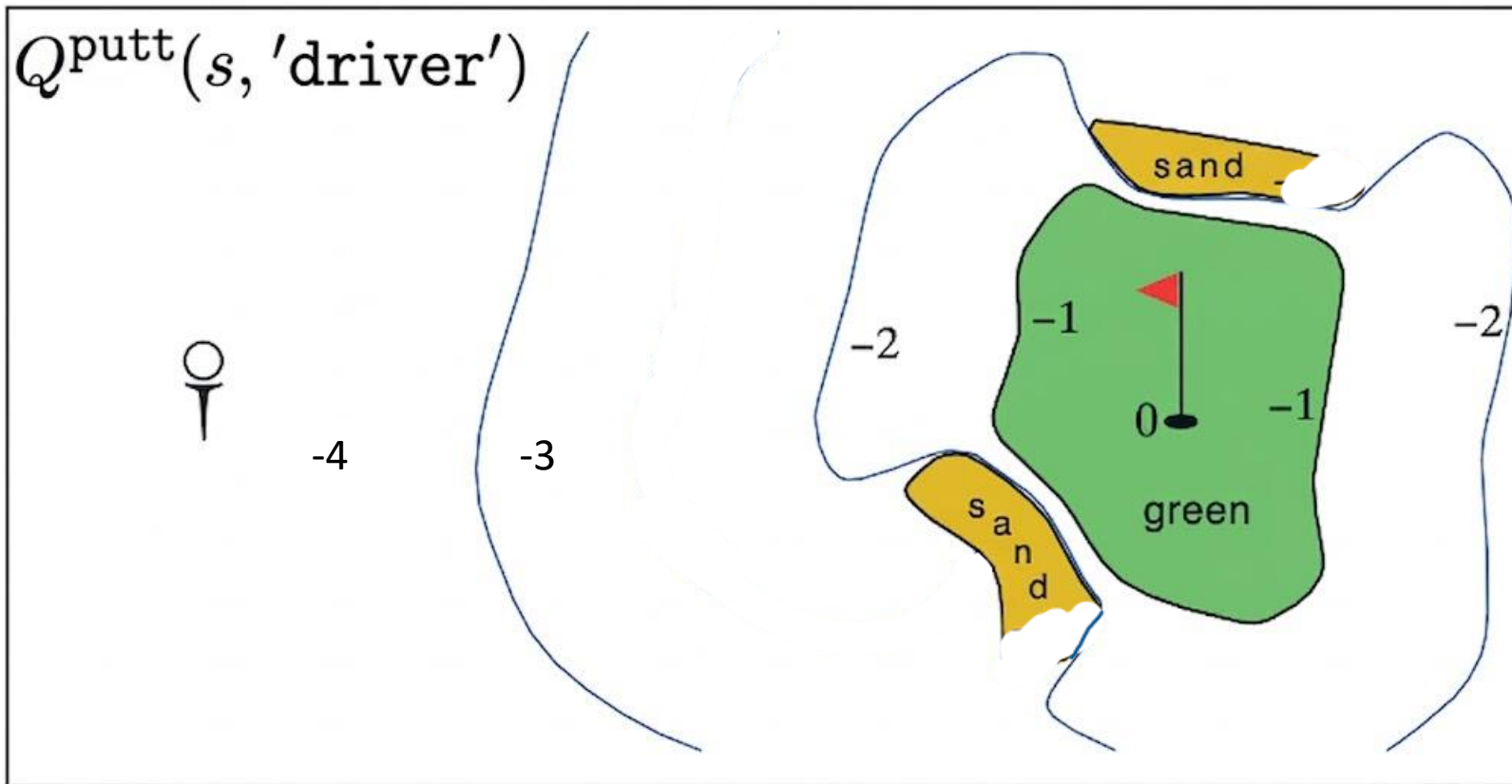
- Optimize for just the next *step*, not the entire trajectory. After taking the “greedy” action a , we still follow π .

$$\pi'(a|s) = \begin{cases} 1 & \text{if } a = \arg \max_{a'} r(s, a') + \gamma \sum_{s' \in S} P_{ss'}^{a'} V^\pi(s') \\ 0 & \text{otherwise} \end{cases}$$

- How to get rid of the model $P_{ss'}^{a'}$?

$Q^\pi(s, a)$ Refresher

$$Q^\pi(s, a) = r(s, a) + \gamma \mathbb{E}_{\mathbb{P}(s_{t+1}|s)}[V^\pi(s_{t+1})|s, a]$$



$$r(s, a) = -1$$



$$\pi(s) = \text{Putt}$$

- $Q^\pi(s, a)$:

First you take action a , then always use the putt

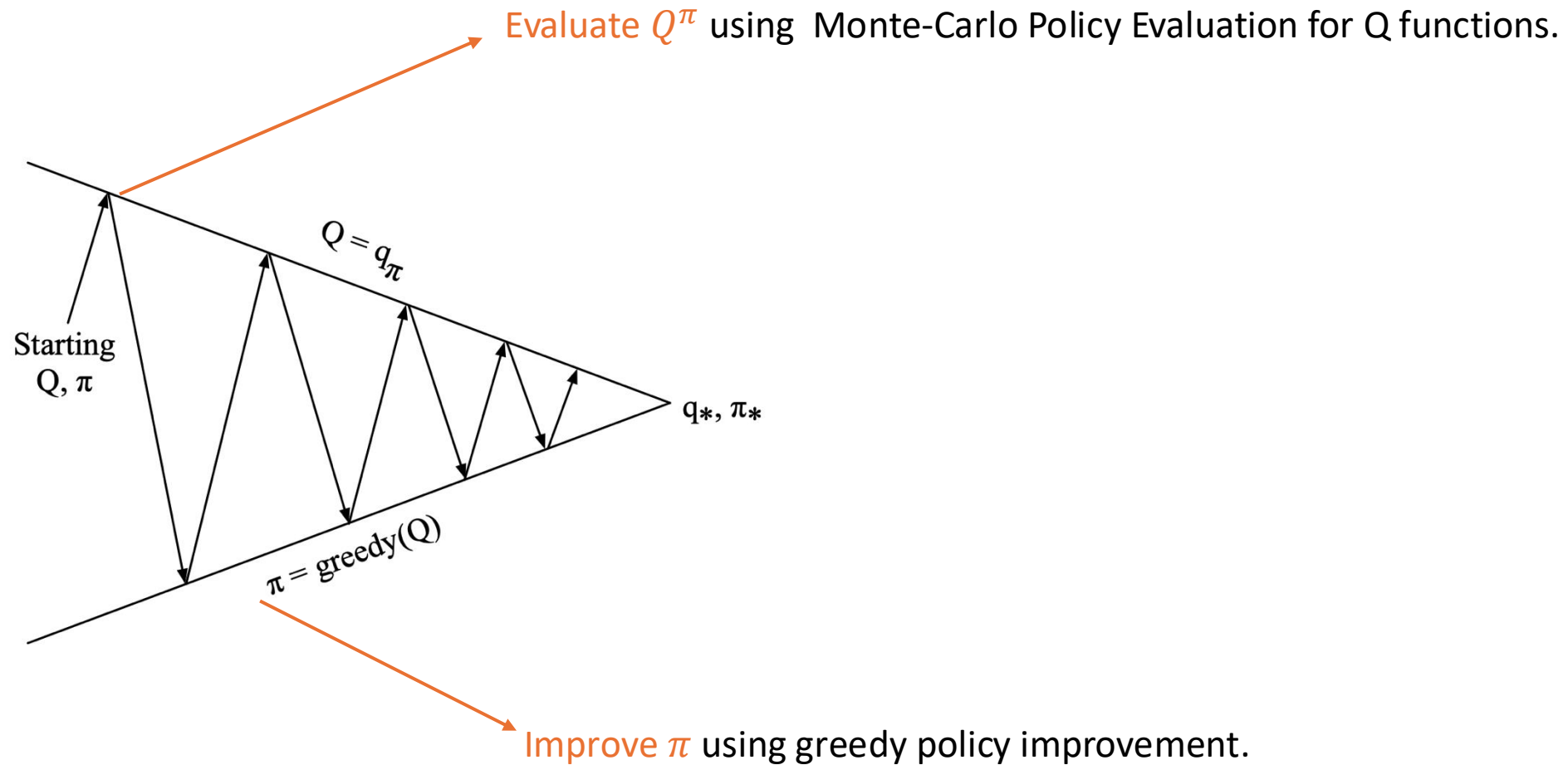
Model-Free Policy Optimization

- $Q^\pi(s, a) = r(s, a) + \gamma \mathbb{E}_{\mathbb{P}(s_{t+1}|s)}[V^\pi(s_{t+1})|s, a]$
- If somebody gives you $Q^\pi(s, a)$, how to find the greedy action?

$$\pi'(a|s) = \begin{cases} 1 & \text{if } a = \arg \max_{a'} Q^\pi(s, a') \\ 0 & \text{otherwise} \end{cases}$$

- Model-free policy optimization 😊

Monte-Carlo Policy Iteration With Q function



However...

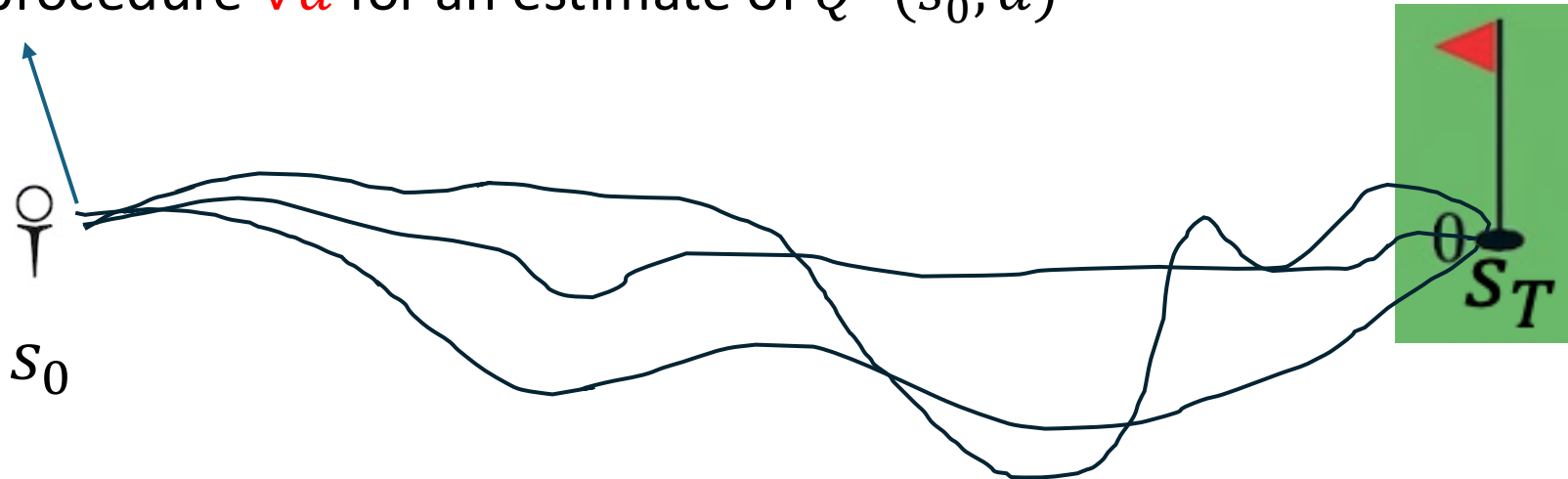
- Estimating Q^π requires many more samples than V^π

$$r(s, a) = -1$$



$$\pi(s) = \text{Putt}$$

Repeat the procedure $\forall a$ for an estimate of $Q^\pi(s_0, a)$



However...

- Estimating Q^π requires many more samples than V^π
- Monte-Carlo Policy Iteration with Q function does NOT converge to Q^*



Example of Greedy Action Selection



“Behind one door is tenure, behind the other is flipping burgers at McDonald’s”

- There are two doors in front of you.
- You open the left door and get reward 0
 - $V(left) = 0$
- You open the right door and get reward 1
 - $V(right) = 1$
- You open the right door and get reward 3
 - $V(right) = 3$
- You open the right door and get reward 2
 - $V(right) = 2$
- **What door would you open next?**

ϵ -Greedy Exploration

- Simplest idea for ensuring continual exploration
- All m actions are tried with non-zero probability
- With probability $1 - \epsilon$ choose the greedy action
- With probability ϵ choose an action at random

$$\pi(a|s) = \begin{cases} \frac{\epsilon}{m} + 1 - \epsilon & \text{if } a^* = \operatorname{argmax}_{a \in A} Q(s, a) \\ \frac{\epsilon}{m} & \text{otherwise} \end{cases}$$

Refresher: Weighted Mean

- The weighted mean of **X** with respect to **W** is:

$$\mu^w = \frac{\sum_{n=1}^N w_n x_n}{\sum_{n=1}^N w_n}$$

- Theorem:

$$\mu^w = \frac{\sum_{n=1}^N w_n x_n}{\sum_{n=1}^N w_n} \leq \max_n(x_1, \dots, x_N)$$

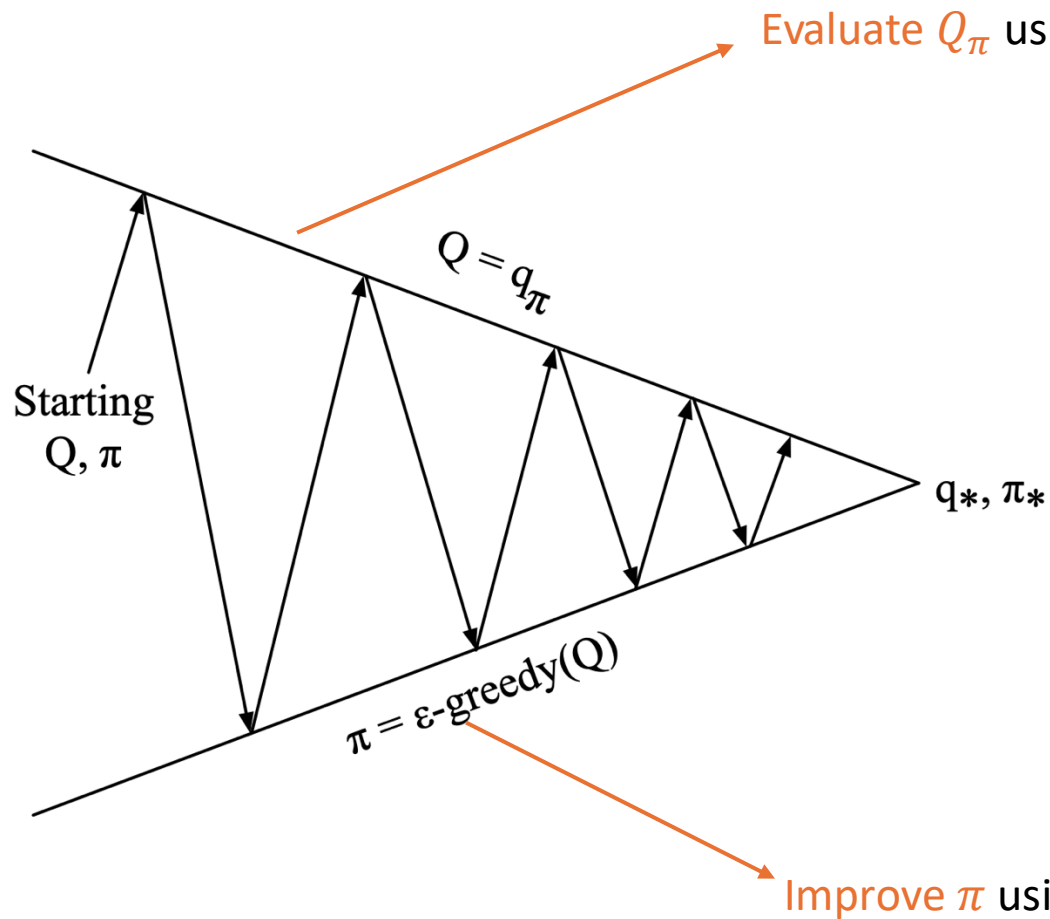
ϵ -Greedy Policy Improvement

- *Theorem.* For any ϵ -greedy policy π , the ϵ -greedy policy π' with respect to Q^π is an *improvement*, $V^{\pi'}(s) \geq V^\pi(s), \forall s$

- *Proof.* (On blackboard)

$$\begin{aligned} V^{\pi'}(s) &= Q^\pi(s, \pi'(s)) = \sum_{a \in A} \pi'(a|s) Q^\pi(s, a) \\ &= \frac{\epsilon}{m} \sum_{a \in A \setminus a^*} Q^\pi(s, a) + \left(1 - \epsilon + \frac{\epsilon}{m}\right) \max_{a \in A} Q^\pi(s, a) \\ &\geq \frac{\epsilon}{m} \sum_{a \in A} Q^\pi(s, a) + (1 - \epsilon) \sum_{a \in A} \frac{\pi(a|s) - \frac{\epsilon}{m}}{1 - \epsilon} Q^\pi(s, a) \\ &= \sum_{a \in A} \pi(a|s) Q^\pi(s, a) = V^\pi(s) \end{aligned}$$

Monte-Carlo Policy Iteration With Q function



Evaluate Q_π using Monte-Carlo Policy Evaluation for Q functions.



This algorithm converges to the optimal policy!

Improve π using ϵ -greedy policy improvement.

MC Policy Iteration with Q fn for Golf

$s \in S$

- Step 1: Use a lot of episodes to learn $Q^\pi(s, a)$.
- Step 2: $\pi \leftarrow \epsilon\text{-greedy}(Q^\pi(s, a))$
- Step 3: Use a lot of episodes to learn $Q^\pi(s, a)$.
- ...



s_0

- Terribly sample inefficient!
- We're collecting a lot of episodes every time the policy changes.

$a \in A$



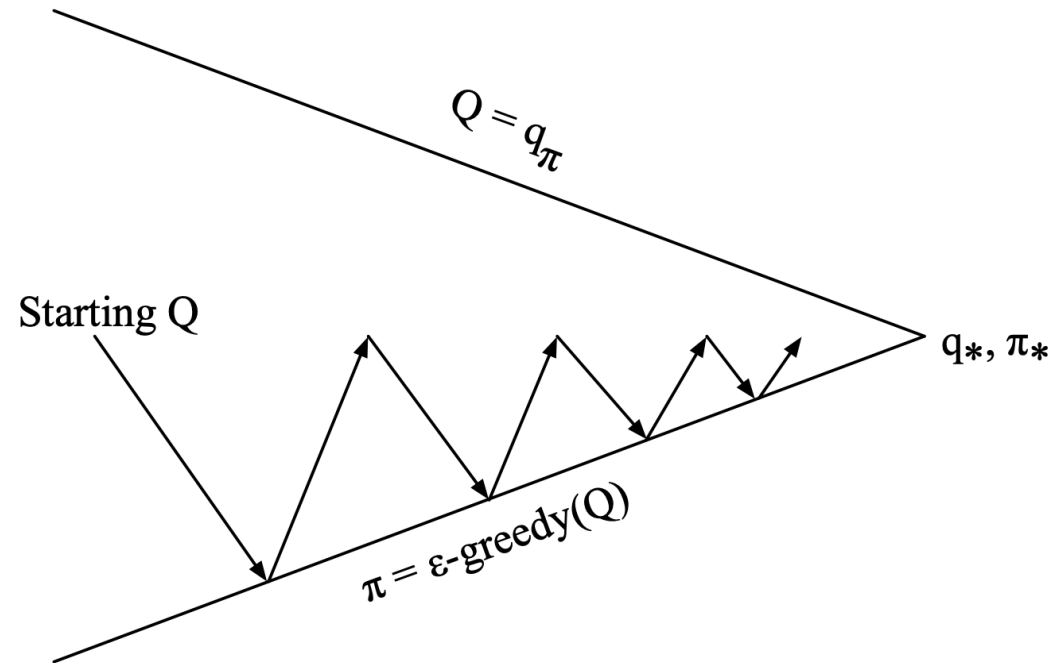
Driver



Putt

$$r(s, a) = -1$$

Single Episode Monte-Carlo Control



- **Every Episode:**
 - Policy Evaluation:
Online Monte-Carlo Policy evaluation
 $Q \approx Q^\pi$ on the episode's data.
 - Policy Improvement:
 ϵ -greedy policy improvement on the latest Q .
- We can prove that this converges to the optimal policy!

GLIE

- *Definition.* Greedy in the Limit with Infinite Exploration (GLIE) if
 - *All state-action pairs are explored infinitely many times,*

$$\lim_{k \rightarrow \infty} N_k(s, a) = \infty$$

- *The policy converges on a greedy policy,*

$$\lim_{k \rightarrow \infty} \pi_k(a|s) = \mathbf{1}(a = \operatorname{argmax}_{a' \in A} Q_k(s, a'))$$

- ϵ -greedy is GLIE if ϵ reduces to zero at $\epsilon_k = \frac{1}{k}$

GLIE Monte-Carlo Control

- Sample kth episode using $\pi: \{s_1, a_1, r_1, \dots\} \sim \pi$

- For each state and action in the episode,

$$N(s_t, a_t) \leftarrow N(s_t, a_t) + 1$$

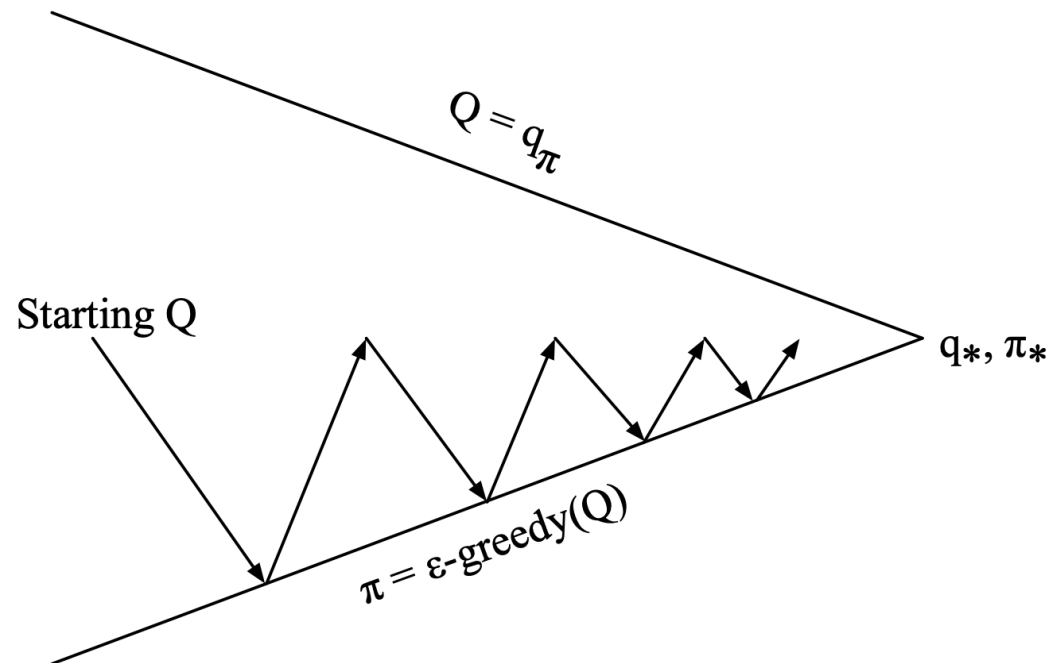
$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \frac{1}{N(s_t, a_t)} (G_t - Q(s_t, a_t))$$

- Improve policy based on new Q function,

$$\epsilon \leftarrow \frac{1}{k}, \quad \pi \leftarrow \epsilon\text{-greedy}(Q)$$

- *Theorem.* GLIE Monte-Carlo control converges to the optimal Q function, $Q(s, a) \leftarrow Q^*(s, a)$.

Using different Policy Evaluation Methods



- Instead of MC, we could use temporal difference (TD) methods for Q function evaluation:
 - TD(0)
 - TD(λ)
- TD has advantages wrt MC:
 - Lower variance.
 - Online.
 - It can learn from incomplete sequences.

Model-Free Control with TD(0) Evaluation

Algorithm Summary:

- Init $Q(s,a)$
- Until tired:
 - Deploy the policy π in the environment to collect an episode $(s_0, a_1, r_1, s_1, a_2, r_2, \dots, s_T)$
 - Apply TD(0) to $Q(s, a)$
 - Use ϵ -greedy policy improvement to obtain a new π

Learning a Q function with TD(0)

- Refresher: Evaluating $V(s)$ with TD(0):

$$V(s) \leftarrow V(s) + \alpha(r(s, a) + \gamma V(s') - V(s))$$

- Deploying a policy in the environment for a single step gives us (s, a, r, s') , so we have all the sufficient data to make the TD backup.
- Let's write the equivalent formula for evaluating $Q(s, a)$ with TD(0):

$$Q(s, a) \leftarrow Q(s, a) + \alpha(r(s, a) + \gamma Q(s', a') - Q(s, a))$$

- What to use as next action?

Learning a Q function with TD(0)

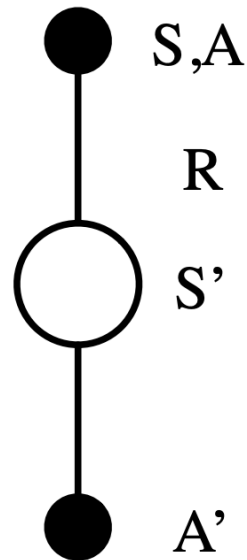
$$Q(s, a) \leftarrow Q(s, a) + \alpha(r(s, a) + \gamma Q(s', a') - Q(s, a))$$

- How to get a' ?
- Option 1 (SARSA): Sample from the current policy
$$a' = \pi(\cdot | s')$$
- Option 2 (Q-Learning): Sample the greedy action at the next step
$$a' = \operatorname{argmax}_a Q(s', a)$$

Learning a Q function with TD(0)

- Update the Q function with TD-learning

$$Q(s, a) \leftarrow Q(s, a) + \alpha(r(s, a) + \gamma Q(s', a') - Q(s, a))$$



SARSA!



Model-Free Control with TD(0) Evaluation

Algorithm Summary:

- Init $Q(s,a)$
- Until tired:
 - Deploy the policy π in the environment to collect an episode $(s_0, a_1, r_1, s_1, a_2, r_2, \dots, s_T)$
 - Apply TD(0) to $Q(s, a)$
 - Use ϵ -greedy policy improvement to obtain a new π

ϵ -Greedy Policy Improvement

$$\pi(a|s) = \begin{cases} \frac{\epsilon}{m} + 1 - \epsilon & \text{if } a^* = \operatorname{argmax}_{a \in A} Q(s, a) \\ \frac{\epsilon}{m} & \text{otherwise} \end{cases}$$

Latest Q function

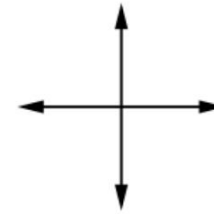
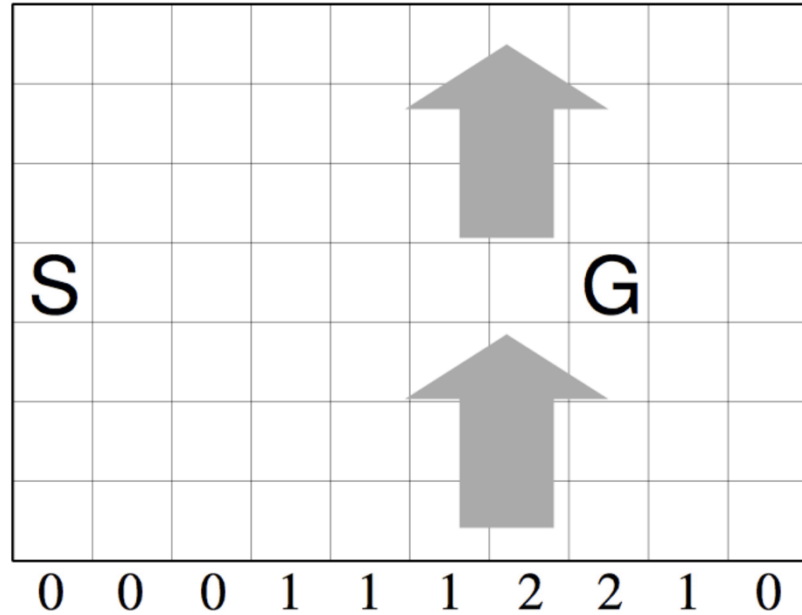


Convergence of Sarsa

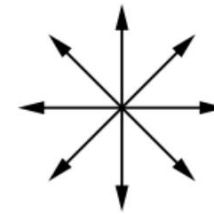
- *Theorem*. Sarsa converges to the optimal Q function, $Q(s, a) \leftarrow Q^*(s, a)$, under the following conditions:
 1. *GLIE sequence of policies* $\pi_t(a|s)$
 2. Robbins-Monro sequence of step-sizes α_t

$$\sum_{t=1}^{\infty} \alpha_t = \infty$$
$$\sum_{t=1}^{\infty} \alpha_t^2 < \infty$$

Windy Gridworld Example



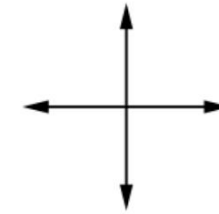
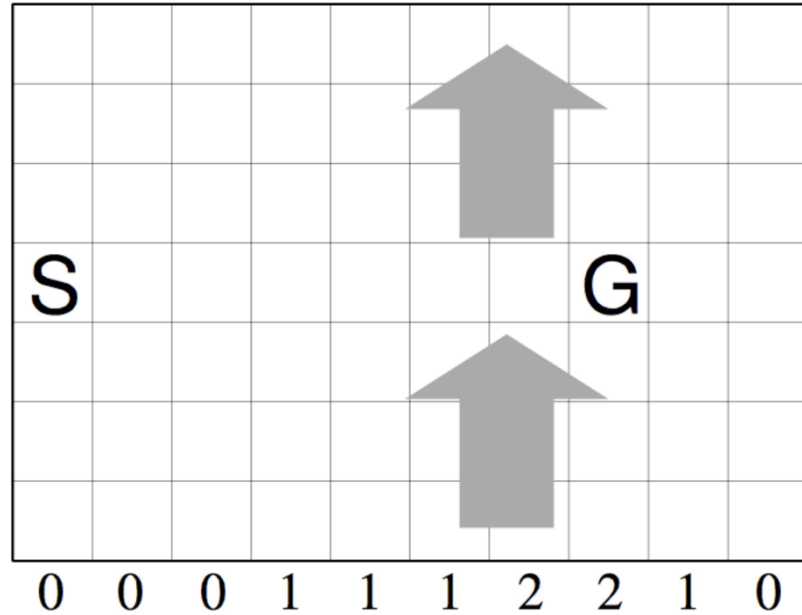
standard
moves



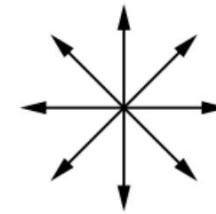
king's
moves

- Reward = -1 per time-step until reaching goal
- Undiscounted

Windy Gridworld Example



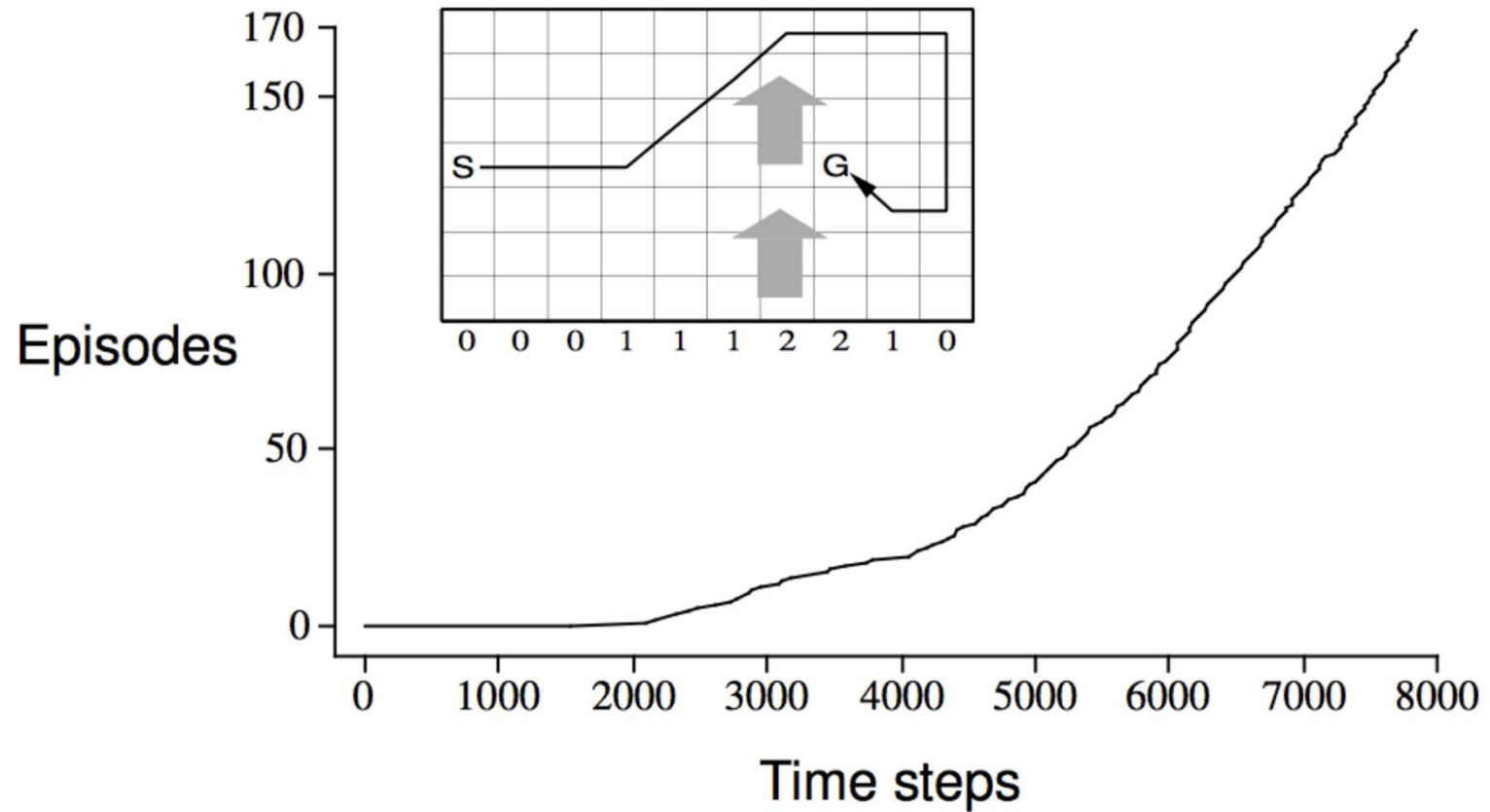
standard
moves



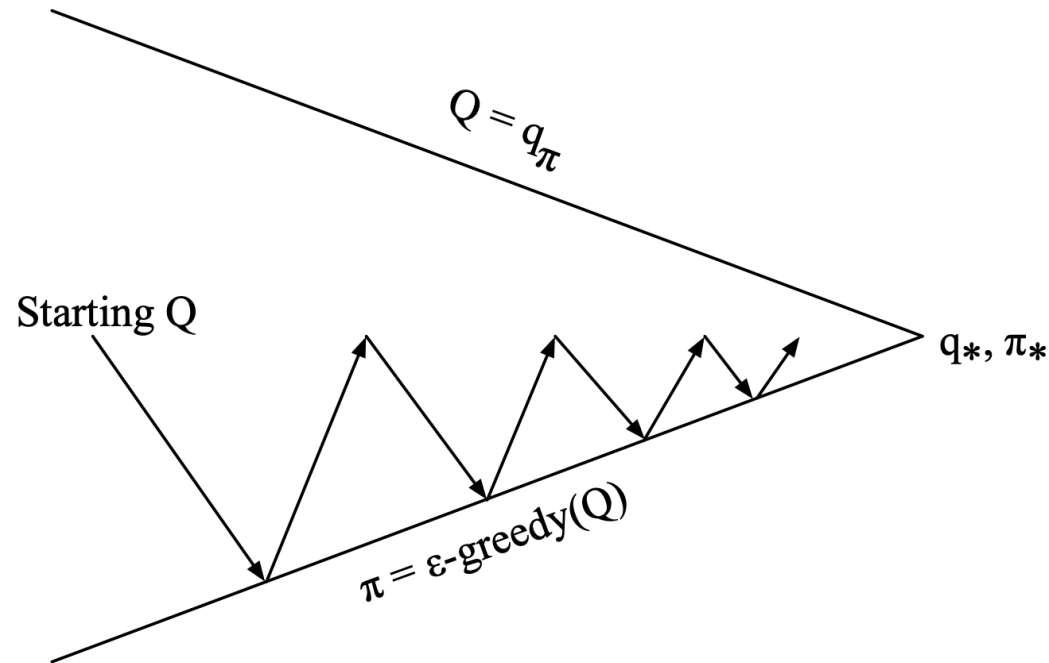
king's
moves

- Cannot easily use MC for evaluation: Not all episodes terminate.
- TD methods are better suited because they learn from every step.

Sarsa on Windy Gridworld



Using different Policy Evaluation Methods



- Instead of MC, we could use temporal difference (TD) methods for Q function evaluation:
 - TD(0)
 - TD(λ)

N-Step Sarsa

- Consider the following n-step returns for $n = 1, 2, \infty$

$$n = 1 \quad (\text{Sarsa}) \quad Q_t^{(1)} = r_{t+1} + \gamma Q(S_{t+1}, A_{t+1})$$

$$n = 2 \quad Q_t^{(2)} = r_{t+1} + \gamma r_{t+2} + \gamma^2 Q(S_{t+2}, A_{t+2})$$

$$\vdots \quad \vdots$$

$$n = \infty \quad (MC) \quad Q_t^{(\infty)} = r_{t+1} + \gamma r_{t+2} + \dots + \gamma^{T-1} r_T$$

- Define the n-step Q-return:

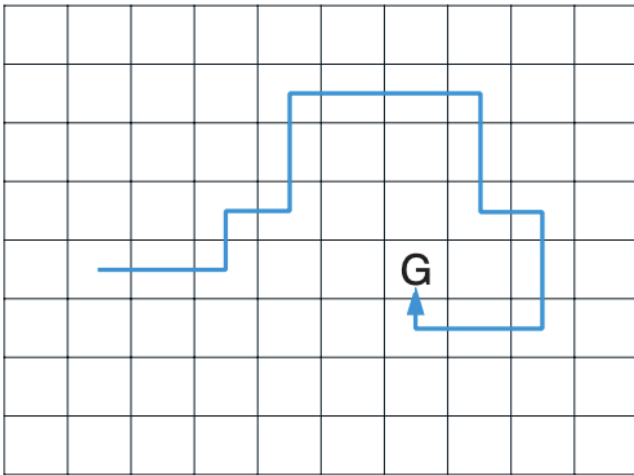
$$Q_t^{(n)} = r_{t+1} + \gamma r_{t+2} + \dots + \gamma^{n-1} r_{t+n} + \gamma^n Q(s_{t+n}, a_{t+n})$$

- N-step Sarsa updates $Q(s, a)$ towards the n-step Q-return

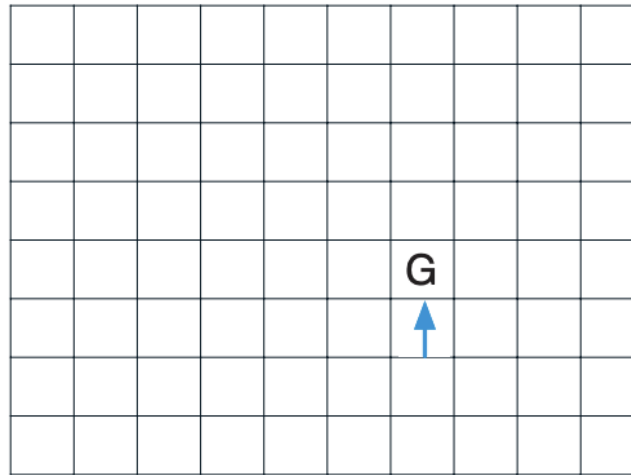
$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha \left(Q_t^{(n)} - Q(s_t, a_t) \right)$$

Sarsa(n) Gridworld Example

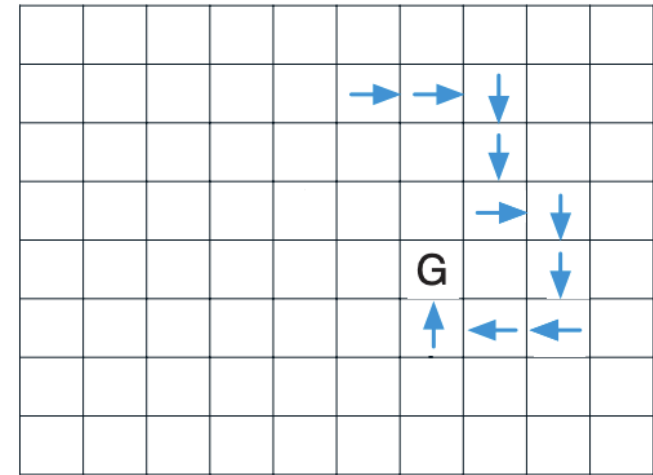
Path taken



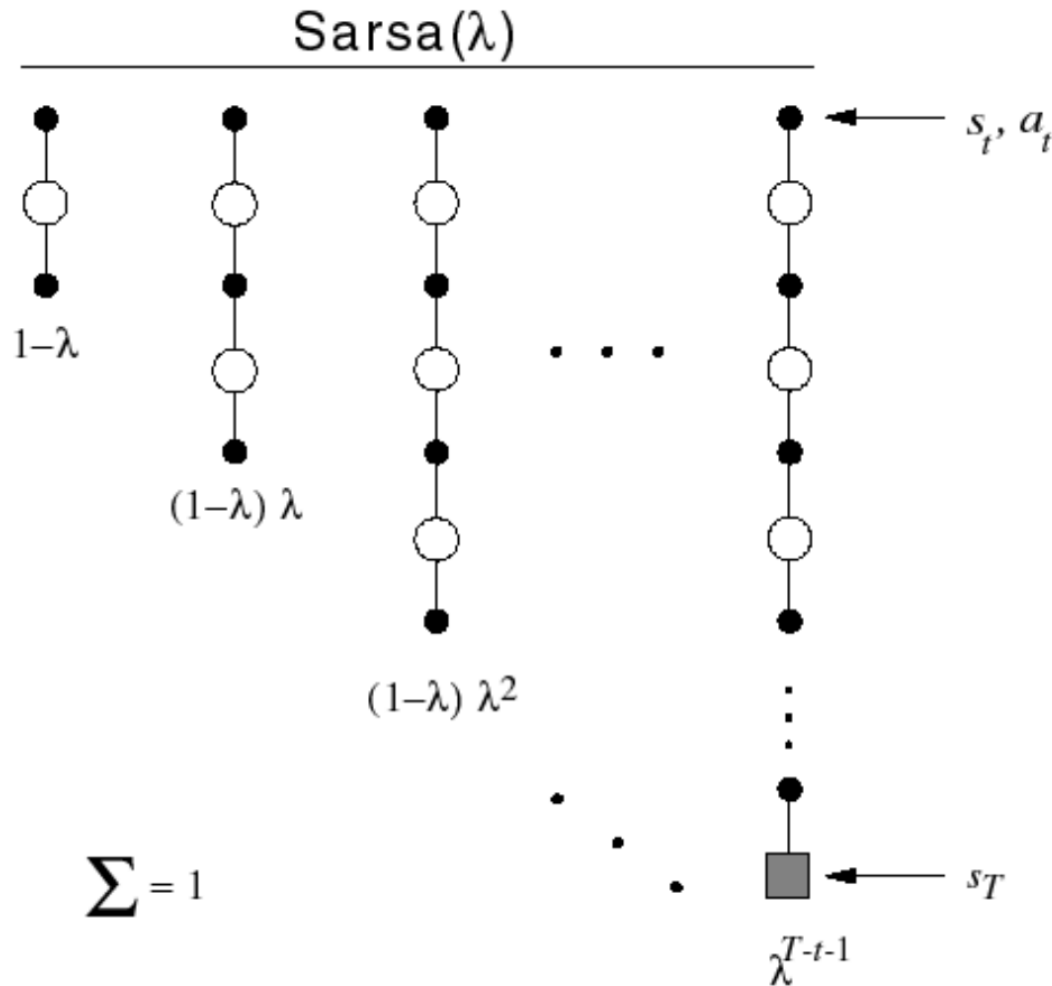
Action values increased
by one-step Sarsa



Action values increased
by 10-step Sarsa



Forward View Sarsa(λ)



- The Q^λ return *combines* all n-step Q-returns $Q_t^{(n)}$

- Using *weight* $(1-\lambda)\lambda^{n-1}$

$$Q_t^\lambda = (1-\lambda) \sum_{n=1}^{\infty} \lambda^{n-1} Q_t^{(n)}$$

- Forward-view Sarsa(λ)

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha \left(Q_t^\lambda - Q(s_t, a_t) \right)$$

Learning a Q function with TD(0)

$$Q(s, a) \leftarrow Q(s, a) + \alpha(r(s, a) + \gamma Q(s', a') - Q(s, a))$$

- How to get a' ?
- Option 1 (SARSA): Sample from the current policy
$$a' = \pi(\cdot | s')$$
- Option 2 (Q-Learning): Sample the **greedy** action at the next step
$$a' = \operatorname{argmax}_a Q(s', a)$$

Useful Identity

- If $a'_g = \operatorname{argmax}_{a'} Q(s', a')$

$$\begin{aligned} Q(s', a'_g) &= Q\left(s', \operatorname{argmax}_{a'} Q(s', a')\right) \\ &= \max_{a'} Q(s', a') \end{aligned}$$

Q-learning Control Algorithm (Watkin, 89)

$$Q(s, a) \leftarrow Q(s, a) + \alpha \left(r + \gamma \max_{a'} Q(s', a') - Q(s, a) \right)$$

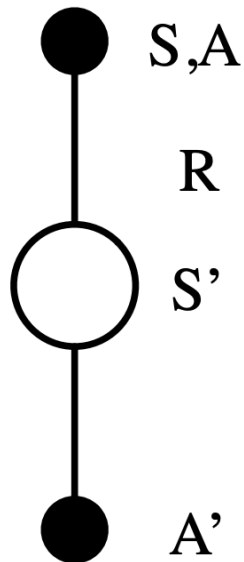
- In contrast to Sarsa, this is **not** attempting to learn Q^π , but Q^*
 - Because the next action is not selected according to π (which is ϵ -greedy)
- Example of an *off-policy* algorithm
 - The policy used to collect experience (π ϵ -greedy)
 - differs from the one for which we want to predict Q or V (π^*)

Q-learning Control Algorithm (Watkin, 89)

- *Theorem*. Q-learning control converges to the optimal action-value function, $Q(s, a) \rightarrow Q^*(s, a)$
- Similar hypothesis and proof as in Sarsa
 - All states are visited infinitely many times
 - Conditions on the sequence of step-size parameters α_k

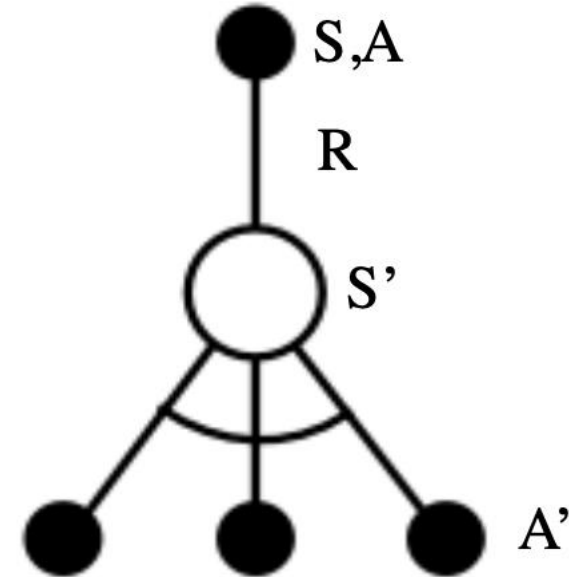
Sarsa vs Q-Learning

SARSA – On Policy



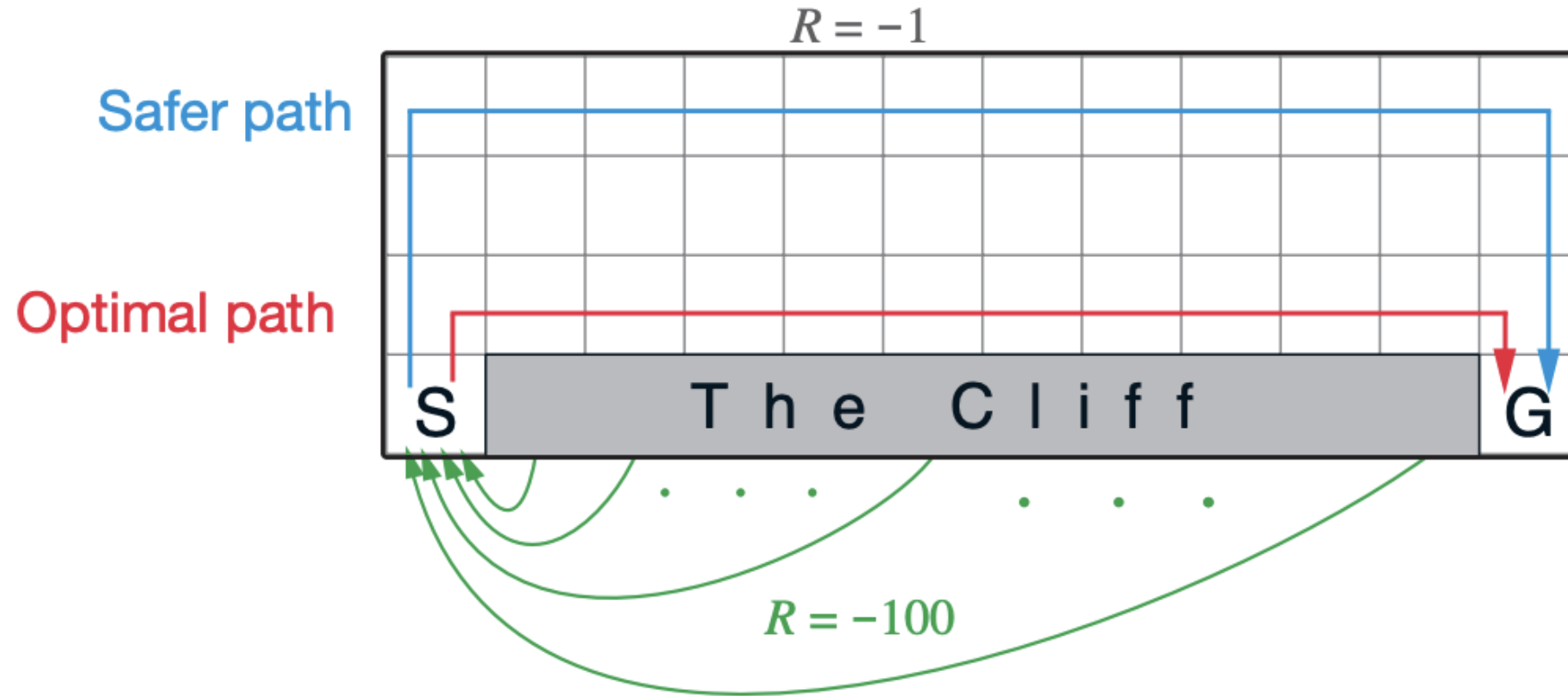
$$\begin{aligned} &Q(s, a) \\ &\leftarrow Q(s, a) \\ &+ \alpha \left(r + \gamma Q(s', \pi(s')) - Q(s, a) \right) \end{aligned}$$

Q-Learning – Off Policy

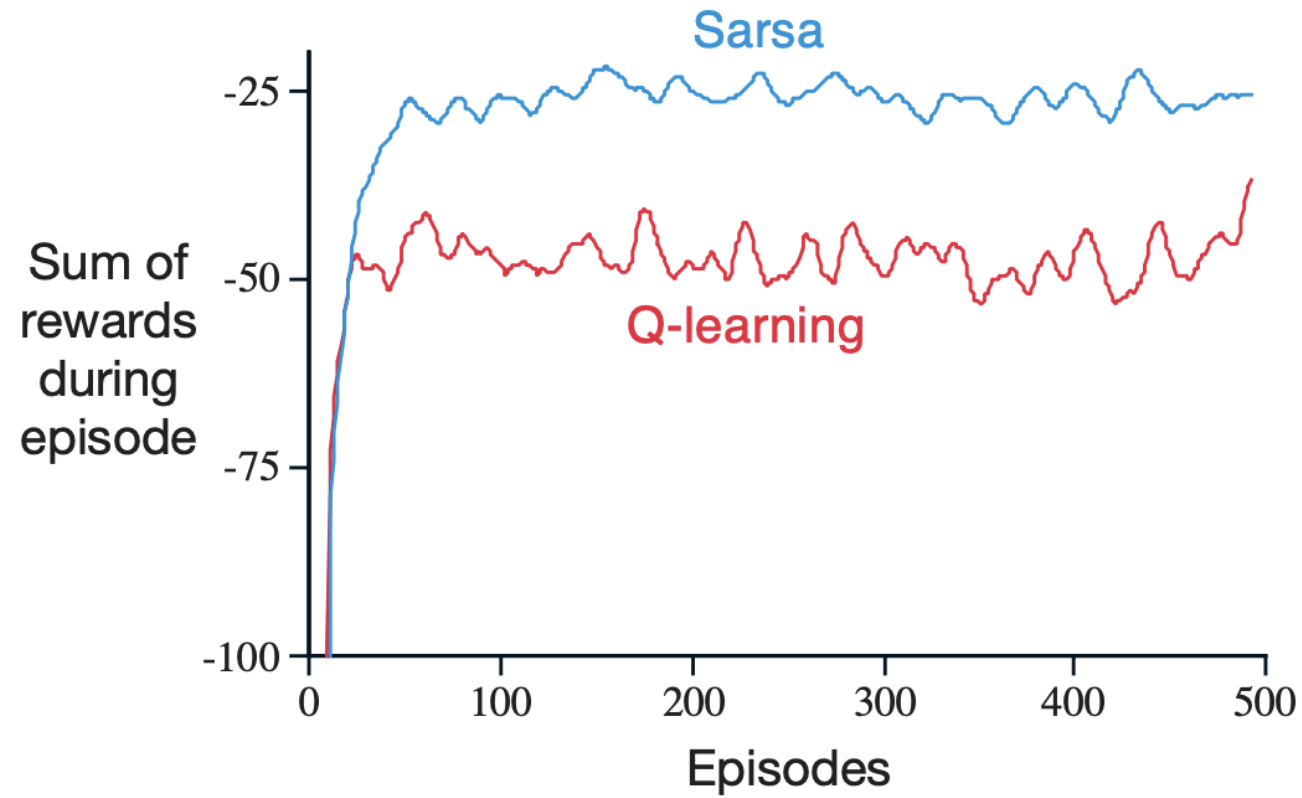


$$\begin{aligned} &Q(s, a) \\ &\leftarrow Q(s, a) \\ &+ \alpha \left(r + \gamma \max_{a'} Q(s', a') - Q(s, a) \right) \end{aligned}$$

Cliff Walking Example



Cliff Walking Example



Expected SARSA

- Similar to Q-learning, but averaging instead of maximizing

- Q-Learning

$$Q(s, a) \leftarrow Q(s, a) + \alpha \left(r + \gamma \max_{a'} Q(s', a') - Q(s, a) \right)$$

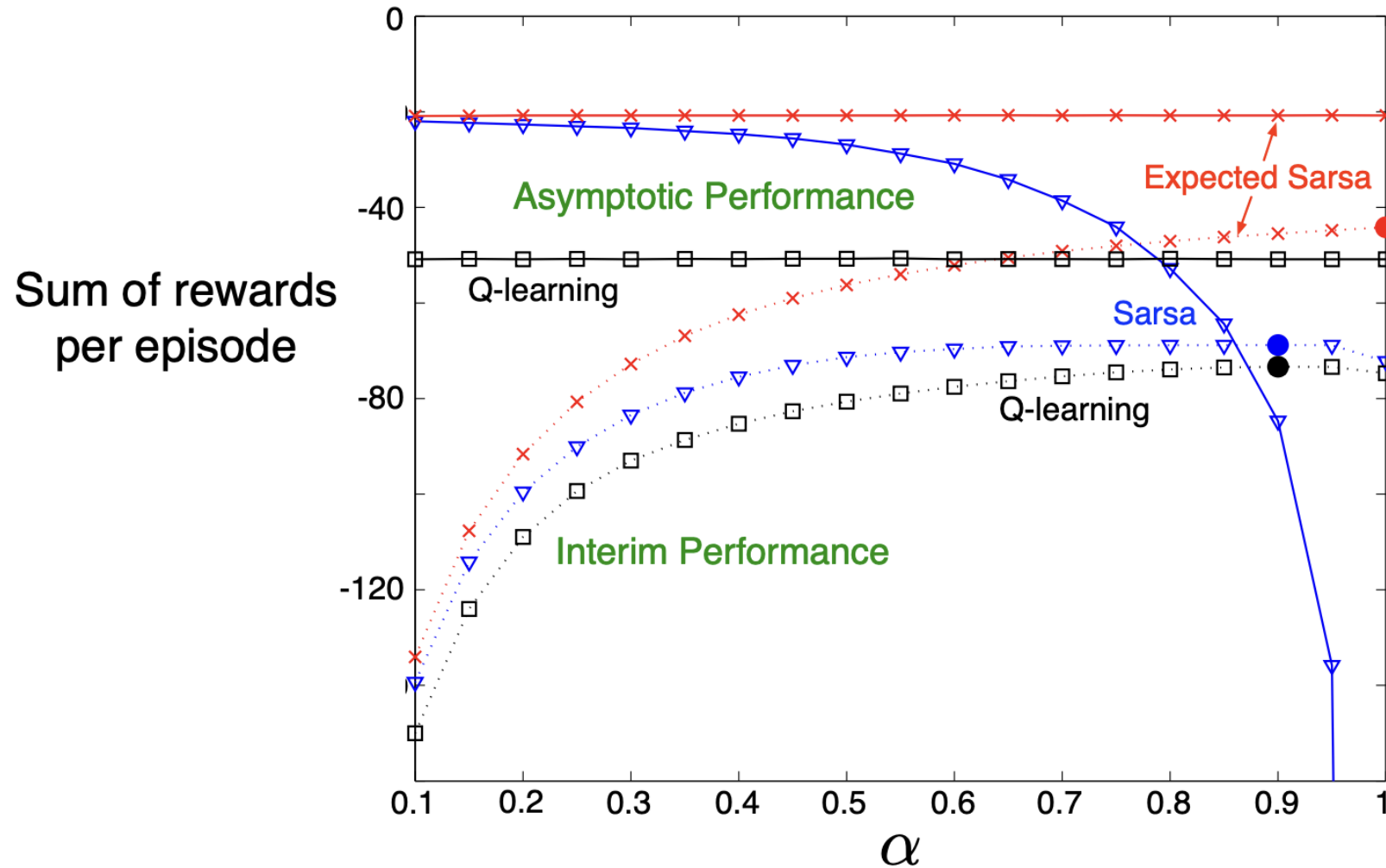
- Expected SARSA

$$Q(s, a) \leftarrow Q(s, a) + \alpha \left(r + \gamma \sum_{a'} \pi(a'|s') Q(s', a') - Q(s, a) \right)$$

Expected SARSA

- Similar updates to SARSA *in expectation*
 - You could get the same update if you sample many times.
- More computationally complex than SARSA because you need to evaluate $Q(s', a') \forall a'$
- In return, it eliminates the variance coming from sampling the next action

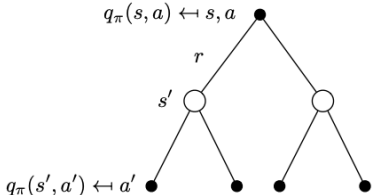
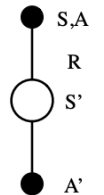
Cliff Walking Example



A Unified View of Control

- We have seen the following methods for control (i.e., finding Q^* and π^*) in the last few classes:
 - Dynamic Programming (Q-Policy Iteration/Q Iteration)
 - Monte-Carlo Model-free Control
 - Sarsa
 - Q-learning
- All methods share the following characteristics
 - They maintain an approximate Q function $Q(s, a), \forall s, a$
 - They “backup” values according to *expected* or *optimal* state trajectories
$$Q(s, a) \leftarrow Q(s, a) + \text{Error}(s, a)$$
 - They act (ϵ -) greedily with respect to the current estimate of $Q(s, a)$

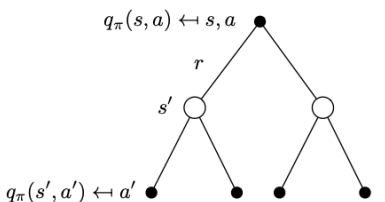
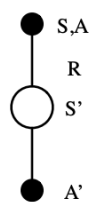
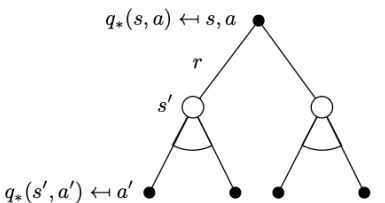
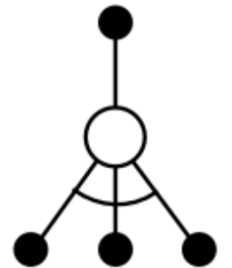
Expectation Update

	<i>Full Backup (DP)</i>	<i>Sample Backup (TD)</i>
Bellman Expectation Equation for $q_\pi(s, a)$	 Q-Policy Iteration	 Sarsa

$$\begin{aligned}
 &Q_{k+1}(s, a) \\
 &= r(s, a) + \gamma \sum_{s' \in S} P_{ss'}^a \sum_{a' \in A} \pi(a'|s') Q_k(s', a')
 \end{aligned}$$

$$\begin{aligned}
 &Q_{k+1}(s, a) \\
 &= Q_k(s, a) \\
 &\quad + \alpha(r(s, a) + \gamma Q_k(s', a') - Q_k(s, a))
 \end{aligned}$$

Optimality Update

	<i>Full Backup (DP)</i>	<i>Sample Backup (TD)</i>
Bellman Expectation Equation for $q_{\pi}(s, a)$	 Q-Policy Iteration	 Sarsa
Bellman Optimality Equation for $q_*(s, a)$	 Q-Value Iteration	 Q-Learning

$$Q_{k+1}(s, a) = r(s, a) + \gamma \sum_{s' \in S} P_{ss'}^a \max_{a' \in A} Q_k(s', a')$$

$$Q_{k+1}(s, a)$$

$$= Q_k(s, a)$$

$$+ \alpha \left(r(s, a) + \gamma \max_{a' \in A} Q(s', a') - Q_k(s, a) \right)$$

Maximization Bias

- Assume the true $Q^*(s, a) = 0, \forall a$
- Assume our current estimate of $Q_k(s, a) = \mathcal{N}(0, \sigma)$, where σ is the RMSE of our current estimate.
- $\max_a Q^*(s, a) = 0$
- $\max_a Q_k(s, a) = \beta > 0$

Maximization Bias: Q-Learning Update

Sarsa Update

$$Q_{k+1}(s, a) = Q_k(s, a) + \alpha \left(r(s, a) + \gamma \max_{a' \in A} Q(s', a') - Q_k(s, a) \right)$$

- If using $\max_a Q^*(s, a) = 0$

$$Q_{k+1}(s, a) = Q_k(s, a) + \alpha(0 - Q_k(s, a)) < |Q_k(s, a)|$$

- If using $\max_a Q_k(s, a) = \beta > 0$

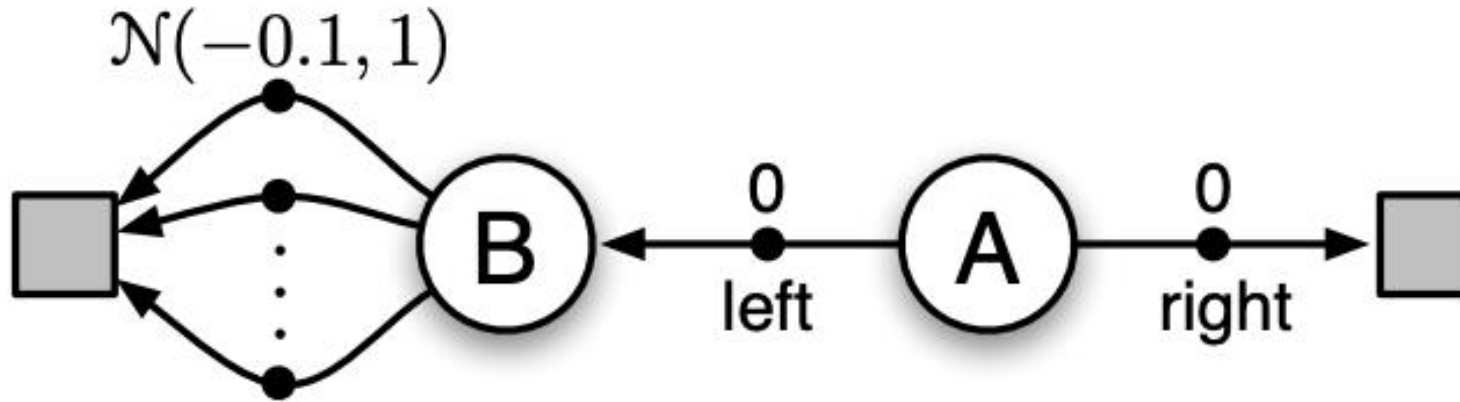
$$Q_{k+1}(s, a) = Q_k(s, a) + \alpha(\gamma\beta - Q_k(s, a)) \geq Q_k(s, a)$$

- The estimated q values are growing!

Maximization Bias

- Even Q_k is a good estimate of Q^*
 - In our case, it was unbiased in expectation
- The maximum over estimated values is generally *larger* than an estimate of the maximum.
- This is called maximization bias
- Not only a problem for Q-learning but also for SARSA (since we do ϵ -greedy action selection).

Example



- $Q^*(A, right) = 0$
- $Q^*(A, left) = V(B) = -0.1$
- When doing Q learning, which action do you think will be most likely sampled?

Maximization Bias: Double Q-Learning

- The problem is that we're estimating the maximum of the true value as the maximum of value estimates.
- This stems from the fact that we're using the same data to:
 - Estimate the maximum action $\widehat{a}^* = \arg \max Q_k(s, a)$
 - Estimate the value of the maximum action $\widehat{Q}^*(s, a) = Q_k(s, \widehat{a}^*)$
- **Idea.** What if we use two different data sources for these goals?
 - Estimate the maximum action $\widehat{a}^* = \arg \max Q_k^1(s, a)$
 - Estimate the value of the maximum action $\widehat{Q}^*(s, a) = Q_k^2(s, \widehat{a}^*)$

Double Q-Learning (Basic)

- Maintain two estimates of Q functions, Q_k^1 and Q_k^2
- At every update step, flip a coin:
 - If tails,
 - $Q_{k+1}^1(s, a) = Q_k^1(s, a) + \alpha \left(r(s, a) + \gamma Q_k^2 \left(s', \arg \max_{a'} Q_k^1(s', a') \right) - Q_k^1(s, a) \right)$
 - $Q_{k+1}^2 = Q_k^2$
 - If heads,
 - $Q_{k+1}^2(s, a) = Q_k^2(s, a) + \alpha \left(r(s, a) + \gamma Q_k^1 \left(s', \arg \max_{a'} Q_k^2(s', a') \right) - Q_k^2(s, a) \right)$
 - $Q_{k+1}^1 = Q_k^1$

Double-Q Learning vs Q-Learning

